University of Fort Hare
*Together in Excellence*

# Residence Maintenance Portal

## Project Documentation

*A web system for reporting and tracking maintenance faults in University of Fort Hare student residences*

### Group 15 members

| # | Name | Student number |
|---|---|---|
| 1 | Abenathi Mgilane | 224018752 |
| 2 | Yomelela Mndela | 202325182 |
| 3 | Liyema Mdandalaza | 225085391 |
| 4 | Aphelele Lutu | 202367973 |
| 5 | Hlalanathi Christopher Nongcawula | 202365450 |
| 6 | Kwanele Mininye | 225031189 |
| 7 | Emihle Dlungwana | 224087536 |
| 8 | Thabile Nomandela | 224027811 |
| 9 | Phumlani Lunga Ngaka | 224017699 |
| 10 | Enathi Nomda | 224031342 |
| 11 | Cebisile Mtyu | 202349002 |
| 12 | Yonela Makaula | 202365204 |
| 13 | Asandiswa Maqam | 202380924 |
| 14 | Philani Ndlovu | 202384756 |

| Item | Details |
|---|---|
| Live system | https://ufh-residence-maintenance.vercel.app |
| Source code | https://github.com/1ungaka/ufh-residence-maintenance |
| Date | September 2026 |

# Contents

1. Introduction ..... 3
2. Problem Statement ..... 3
3. Proposed Solution ..... 4
3.1 Key features ..... 4
4. Specifications ..... 5
4.1 System specifications ..... 5
4.2 Hardware and software requirements ..... 5
5. Languages and Technologies Used ..... 6
5.1 Programming languages ..... 6
5.2 Frameworks, libraries and tools ..... 6
6. Functional Requirements ..... 7
7. Non-Functional Requirements ..... 8
8. Operation Modes and User Modes ..... 9
8.1 Operation modes ..... 9
9. Concept of Operation ..... 10
9.1 Typical scenario ..... 10
9.2 System architecture ..... 10
9.3 Life cycle of a request ..... 11
9.4 Data model ..... 11
10. Use Case Diagram ..... 12
10.1 Use case descriptions ..... 12
11. Activity Diagrams ..... 14
11.1 Student logs a fault ..... 14
11.2 Tradesperson handles a job ..... 15
11.3 Sign in and password reset ..... 16
12. Methodology ..... 17
12.1 Iterations ..... 17
12.2 Team practices ..... 17
13. Demo and Testing ..... 18
13.1 Demonstration plan ..... 18
13.2 Automated tests (prototype backend) ..... 18
13.3 Functional tests of the final system ..... 18
13.4 User acceptance testing ..... 19
14. Conclusion ..... 20
14.1 Limitations and future work ..... 20

# 1. Introduction

The Residence Maintenance Portal is a web system that lets University of Fort Hare (UFH) students report faults in their residence rooms and follow each repair until it is fixed. Maintenance tradespeople use the same system to see which jobs are most urgent, accept them, and keep the student informed.

This document describes the problem the system solves, the solution, its specifications and requirements, how it operates, and how it was built and tested. The system is live at **https://ufh-residence-maintenance.vercel.app**.

# 2. Problem Statement

Students in UFH residences report maintenance problems such as broken lights, blocked drains, faulty locks and leaking taps by walking to the residence office, filling in paper forms, or telling a staff member in passing. This causes several problems:

- **Reports get lost.** Paper forms and verbal reports leave no reliable record, so faults are forgotten and have to be reported again.
- **No visibility for students.** A student cannot see whether anyone has read their report, who is handling it, or when someone will come.
- **No prioritisation.** Maintenance staff receive requests in no particular order, so a room without electricity can wait behind a squeaky cupboard door.
- **Poor communication.** Tradespeople have no simple way to tell a student when they are coming or that they need a part, and students cannot easily give access times.
- **No data for planning.** Without records, Residence Life and Facilities Management cannot see which residences or fault types keep recurring.

> **Problem in one sentence:** there is no single, reliable place where residence faults are reported, prioritised, and tracked from the moment they are logged until they are fixed.

# 3. Proposed Solution

We built a responsive web application, the Residence Maintenance Portal, that works on both phones and computers and serves two kinds of users:

- **Students** sign in with their UFH student email, log a fault (category, residence, room, urgency, description and an optional photo) and follow its progress through a clear history of every update.
- **Maintenance tradespeople** sign in with their staff email, see a work queue ordered by urgency and waiting time, accept jobs, message the student, record progress notes, and mark jobs as completed.

Every change is recorded against the request, so there is always a complete, time-stamped record. Students see updates on their dashboard without having to ask, and tradespeople can see at a glance what to do next.

## 3.1 Key features

- Sign-in with a role choice (Student or Maintenance tradesperson) and checks that the email and account match the chosen role
- Student self-registration; staff accounts issued by Facilities Management
- Password reset by an emailed link that opens the portal directly
- Logging a fault with residence list (Student Village, Marikana, Wesley, ZK, Taunyane, or "Other" with a typed name)
- Photos resized automatically in the browser so any phone photo can be attached
- Request history showing when a job was logged, accepted, started and completed, plus progress notes and student comments
- A work queue ordered by urgency, then by how long the request has waited
- "Latest updates" on the student dashboard, and optional email notifications
- Contact us, Terms and conditions, and a POPIA-based Privacy statement

# 4. Specifications

## 4.1 System specifications

| Item | Specification |
|---|---|
| System type | Responsive web application (single-page application) |
| Users | Students, maintenance tradespeople, Facilities Management (administration) |
| Student email format | `<student number>@ufh.ac.za`, e.g. 202412345@ufh.ac.za |
| Staff email format | `<6-digit staff ID>@staff.ufh.ac.za`, e.g. 123456@staff.ufh.ac.za |
| Password | At least 8 characters; stored hashed by Supabase Auth (never in plain text) |
| Password reset | Emailed link, valid for one hour and usable once |
| Urgency levels | High (unsafe or unusable now), Medium, Low |
| Request statuses | Submitted → Assigned → In Progress → Completed |
| Residences | Student Village, Marikana, Wesley, ZK, Taunyane, or Other (typed) |
| Photos | JPG, PNG or WebP of any size; resized in the browser to at most 1600 px and about 1.5 MB before upload; storage limit 2 MB per photo |
| Supported screens | Phones from 360 px wide up to desktop monitors |

## 4.2 Hardware and software requirements

| For | Requirement |
|---|---|
| Users (students and staff) | Any device with a modern web browser (Chrome, Edge, Firefox or Safari) and an internet connection. No installation needed. |
| Developers | Node.js 18 or newer with npm, Git, and a code editor such as Visual Studio Code |
| Hosting | Vercel (static hosting with automatic deployment from GitHub) |
| Backend services | Supabase: Authentication, PostgreSQL database, Storage and Edge Functions |

# 5. Languages and Technologies Used

## 5.1 Programming languages

| Language | Where it is used |
|---|---|
| JavaScript (ES2022) | All application logic in the frontend: sign-in, validation, data loading, photo resizing |
| JSX (React) | The user interface: pages and reusable components |
| HTML5 | The page shell that loads the application |
| CSS3 | The custom UFH navy-and-gold visual design, layout for phones and computers, animation |
| SQL (PostgreSQL) | Database tables and row-level security policies in Supabase |
| TypeScript (Deno) | The notify-users Supabase Edge Function that emails students |
| JSON | Configuration (package.json, vercel.json) |

## 5.2 Frameworks, libraries and tools

| Technology | Purpose |
|---|---|
| React 18 | Component-based user interface library |
| Vite 5 | Development server and production build tool |
| React Router 6 | Page navigation and protected (sign-in only) routes |
| Supabase JS client | Talks to Supabase Auth, the database and Storage |
| GSAP | The animated photo wall on the sign-in page |
| Git and GitHub | Version control and team collaboration |
| Vercel | Hosting; every push to the main branch redeploys the live site |
| Node.js / npm | Running the development tools and installing packages |

*Note: an earlier prototype backend (Node.js, Express and SQLite, in the backend/ folder) was used during the first iteration. The final system replaced it with Supabase, so the frontend now talks to Supabase directly.*

# 6. Functional Requirements

Functional requirements describe what the system must do.

| ID | Requirement | User |
|---|---|---|
| FR-01 | The system shall let a student register with a UFH student email, full name, residence, room number and a password of at least 8 characters. | Student |
| FR-02 | The system shall require users to choose their role (Student or Maintenance tradesperson) when signing in, check the email format for that role, and refuse sign-in if the account belongs to the other role. | All |
| FR-03 | The system shall let a user reset a forgotten password through a link emailed to their university address. | All |
| FR-04 | The system shall let a student log a fault with a category, residence (from a list or typed under "Other"), room, urgency and description. | Student |
| FR-05 | The system shall let a student attach an optional photo, resize it before upload, and store it in a folder only that student can upload to. | Student |
| FR-06 | The system shall show students only their own requests, with filters by category, status and urgency. | Student |
| FR-07 | The system shall show the full history of a request: logged, accepted, started, completed, progress notes and comments, with who and when. | All |
| FR-08 | The system shall let a student add comments to their own open request. | Student |
| FR-09 | The system shall show the student a dashboard with request counts, recent faults and the latest updates from staff. | Student |
| FR-10 | The system shall show tradespeople a work queue of open requests ordered by urgency and then waiting time, with filters. | Tradesperson |
| FR-11 | The system shall let a tradesperson accept an unassigned job and send the student a message; a job accepted by someone else cannot be changed. | Tradesperson |
| FR-12 | The system shall let the assigned tradesperson move a job to In Progress and then Completed, with an optional message each time. | Tradesperson |
| FR-13 | The system shall let the assigned tradesperson add progress notes that the student can see. | Tradesperson |
| FR-14 | The system shall show each tradesperson a "My jobs" list of jobs assigned to them. | Tradesperson |
| FR-15 | The system shall let users update their profile (name, and residence and room for students). | All |
| FR-16 | The system shall provide Contact us, Terms and conditions and Privacy statement pages that can be read without signing in. | Visitor |
| FR-17 | The system shall be able to email students when their request is created or updated (Supabase Edge Function notify-users). | System |

# 7. Non-Functional Requirements

Non-functional requirements describe how well the system must work.

| ID | Quality | Requirement and how it is met |
|---|---|---|
| NFR-01 | Security | Passwords are hashed by Supabase Auth. All traffic uses HTTPS. Only the public "anon" key is in the frontend; access to data is controlled by row-level security policies. Photo uploads are limited to the user's own folder. |
| NFR-02 | Privacy | Personal information is handled according to the Protection of Personal Information Act (POPIA), explained in the Privacy statement. Students see only their own requests. |
| NFR-03 | Usability | Plain-language labels and messages, a consistent UFH navy-and-gold design, and clear error messages that say how to fix the problem (e.g. which email format to use). |
| NFR-04 | Accessibility | Every input has a label, pages work with the keyboard and show focus outlines, colours have good contrast, and animation is switched off for users who prefer reduced motion. |
| NFR-05 | Responsiveness | Layouts adapt from 360 px phones to large monitors with no sideways scrolling. |
| NFR-06 | Performance | Static files are served from Vercel's global network; photos are shrunk (e.g. 9 MB to about 0.5 MB) before upload so logging a fault is quick on mobile data. |
| NFR-07 | Availability | Hosted on managed services (Vercel and Supabase) that run continuously without a university server. |
| NFR-08 | Maintainability | Shared rules live in one place each (email formats, roles, residences, contact details), and the code is documented in README files. |
| NFR-09 | Portability | Runs in any modern browser on Windows, macOS, Android and iOS; nothing to install. |
| NFR-10 | Reliability | Failed saves show an error instead of pretending to succeed; a change refused by the database is reported as "not allowed". |

# 8. Operation Modes and User Modes

The portal has four user modes. The mode is decided by who is using the system and, after sign-in, by the role stored on their account.

| Mode | Who | What they can do |
|---|---|---|
| **Visitor mode** | Anyone who is not signed in | Sign in, register as a student, reset a password, and read Contact us, Terms and conditions and the Privacy statement. |
| **Student mode** | Students with an @ufh.ac.za account | Dashboard, log a fault, view own requests and their history, comment on open requests, update profile. |
| **Maintenance tradesperson mode** | Staff with an @staff.ufh.ac.za account | Work queue, accept jobs, update status, add progress notes, My jobs, update profile. |
| **Administrator mode** | Facilities Management | Creates and manages staff accounts and system settings through the Supabase dashboard (outside the portal). |

## 8.1 Operation modes

- **Online, live mode:** the deployed system at https://ufh-residence-maintenance.vercel.app, used by students and staff.
- **Development mode:** a developer runs the app on their own computer with npm run dev; it connects to the same Supabase project, so changes can be tried before they are pushed.
- **Continuous deployment:** every push to the main branch on GitHub is built and published by Vercel automatically.

# 9. Concept of Operation

This section describes how the system is used day to day and how its parts work together.

## 9.1 Typical scenario

1. A student in Wesley residence notices that the light in room F29 has stopped working. On their phone they open the portal, choose Student and sign in.
2. They tap Log a fault, choose Electrical, confirm Wesley and room F29, pick High urgency, describe the problem and take a photo. The photo is resized on the phone and uploaded, and the request is saved as Submitted.
3. An electrician signs in as a Maintenance tradesperson. The new High-urgency request appears near the top of the work queue.
4. The electrician accepts the job and writes: "I will come tomorrow between 10:00 and 12:00." The request becomes Assigned, and the message appears in the request history and on the student's dashboard.
5. Next day the electrician starts work (In Progress), finds a faulty fitting, adds a progress note, replaces it, and marks the job Completed with a short message.
6. The student sees the whole history and knows the fault is fixed. The completed request remains as a record for Residence Life.

## 9.2 System architecture

The portal is a React single-page application hosted on Vercel. The browser loads the app once, then talks directly to Supabase over HTTPS for sign-in, data and photos. Supabase row-level security decides what each signed-in user may read or change.

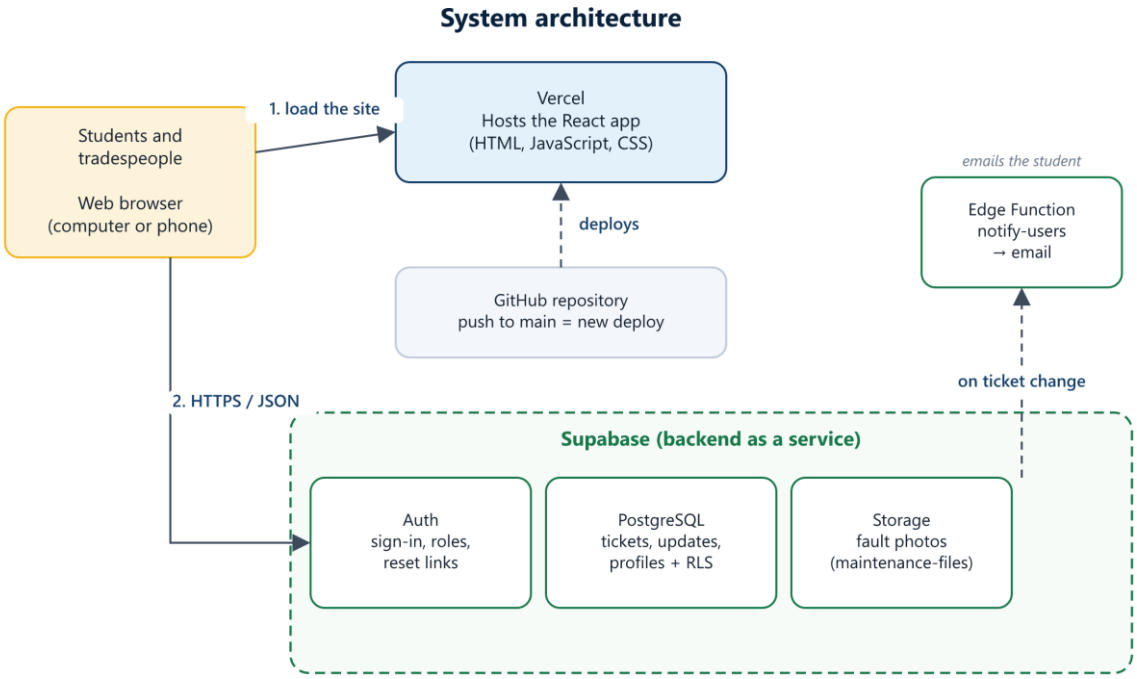



*Figure 1: System architecture*

## 9.3 Life cycle of a request

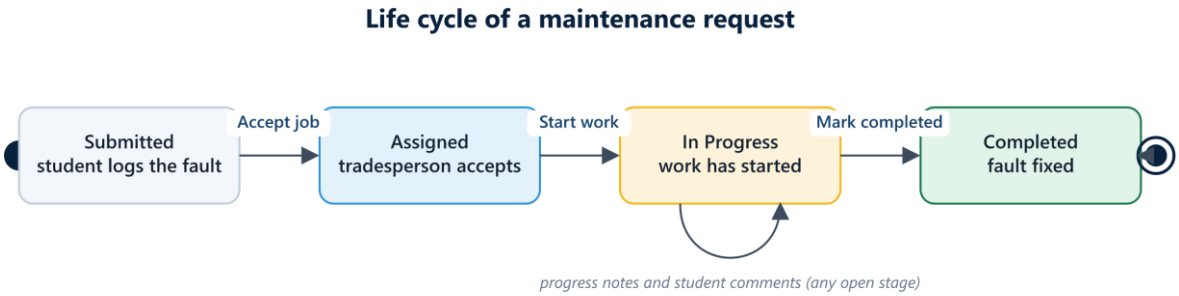



*Figure 2: Status life cycle of a maintenance request*

## 9.4 Data model

The main data is stored in four Supabase (PostgreSQL) tables and one Storage bucket:

| Table / bucket | Holds | Key fields |
|---|---|---|
| profiles | One row per user | id, full_name, role, student_number, residence_building, room_number |
| categories | Fault categories (Electrical, Plumbing, ...) | id, name |
| maintenance_tickets | One row per reported fault | id, reported_by, category_id, residence_building, room_number, urgency, description, status, assigned_to, photo_url, created_at, updated_at |
| ticket_updates | Every entry in a request's history | id, ticket_id, updated_by, update_message, created_at |
| maintenance-files (Storage) | Fault photos | Saved as <user id>/<random id>.jpg |

# 10. Use Case Diagram

The use case diagram shows the actors that interact with the portal and what each can do. Student and Tradesperson are both kinds of Portal user, so they share sign-in, password reset and profile updates (the hollow arrows show this generalisation). Attaching a photo is an optional extension of logging a fault.

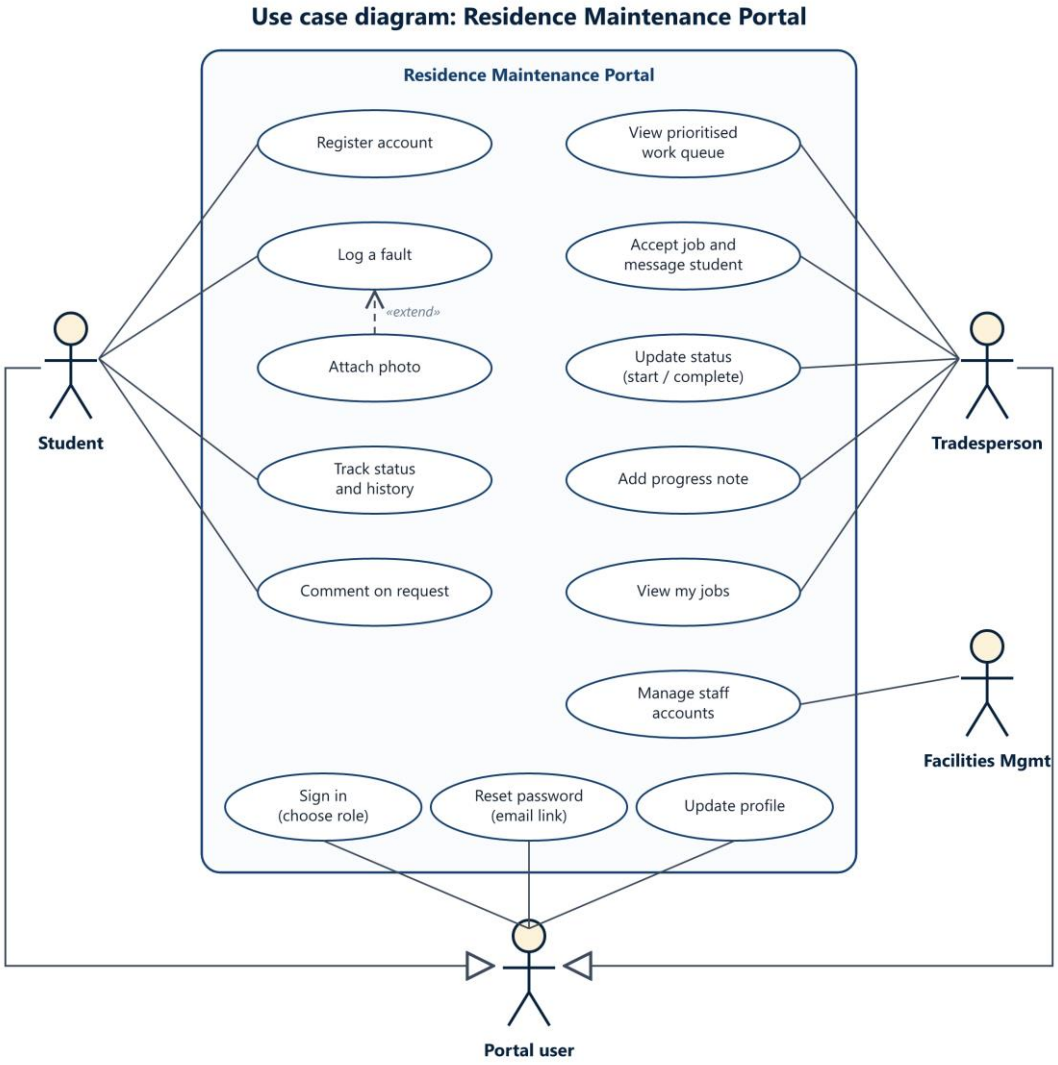



*Figure 3: Use case diagram*

## 10.1 Use case descriptions

| Use case | Actor | Description |
|---|---|---|
| Register account | Student | Creates a student account with a UFH student email, name, residence, room and password. |
| Sign in (choose role) | Portal user | Chooses Student or Tradesperson, enters email and password; the system checks the email format and that the account role matches. |
| Reset password | Portal user | Requests a reset link by email and chooses a new password. |
| Log a fault | Student | Submits category, residence, room, urgency and description. Status starts as Submitted. |

| Use case | Actor | Description |
|---|---|---|
| Attach photo | Student | Optional: adds a photo, which is resized and uploaded to the student's folder. |
| Track status and history | Student | Views own requests and the time-stamped history of each. |
| Comment on request | Student | Adds information for the tradesperson, e.g. times they are in their room. |
| View prioritised work queue | Tradesperson | Sees open requests, most urgent and longest-waiting first. |
| Accept job and message student | Tradesperson | Takes an unassigned job and tells the student when they will come. |
| Update status | Tradesperson | Moves the job to In Progress, then Completed, with an optional message. |
| Add progress note | Tradesperson | Records progress the student can see. |
| View my jobs | Tradesperson | Lists jobs assigned to them. |
| Manage staff accounts | Facilities Mgmt | Creates tradesperson accounts and sets their role in Supabase. |
| Update profile | Portal user | Changes name, and residence and room for students. |

# 11. Activity Diagrams

The activity diagrams show the step-by-step flow of the three most important activities. Rounded boxes are actions, gold diamonds are decisions, the filled circle is the start, and the ringed circle is the end.

## 11.1 Student logs a fault

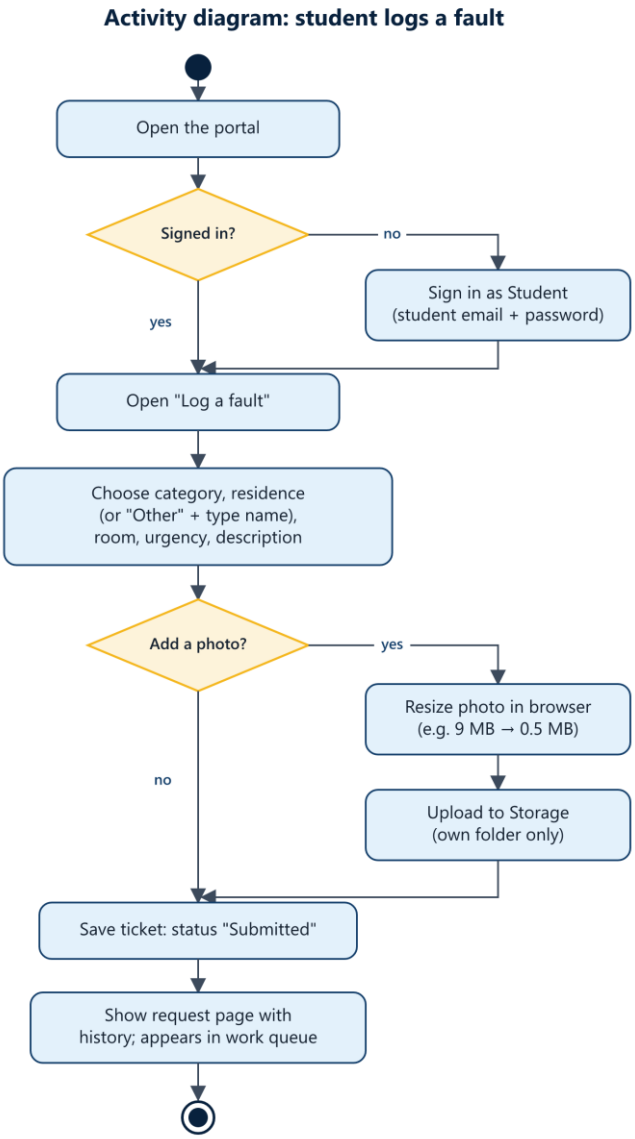



*Figure 4: Activity diagram: student logs a fault*

## 11.2 Tradesperson handles a job

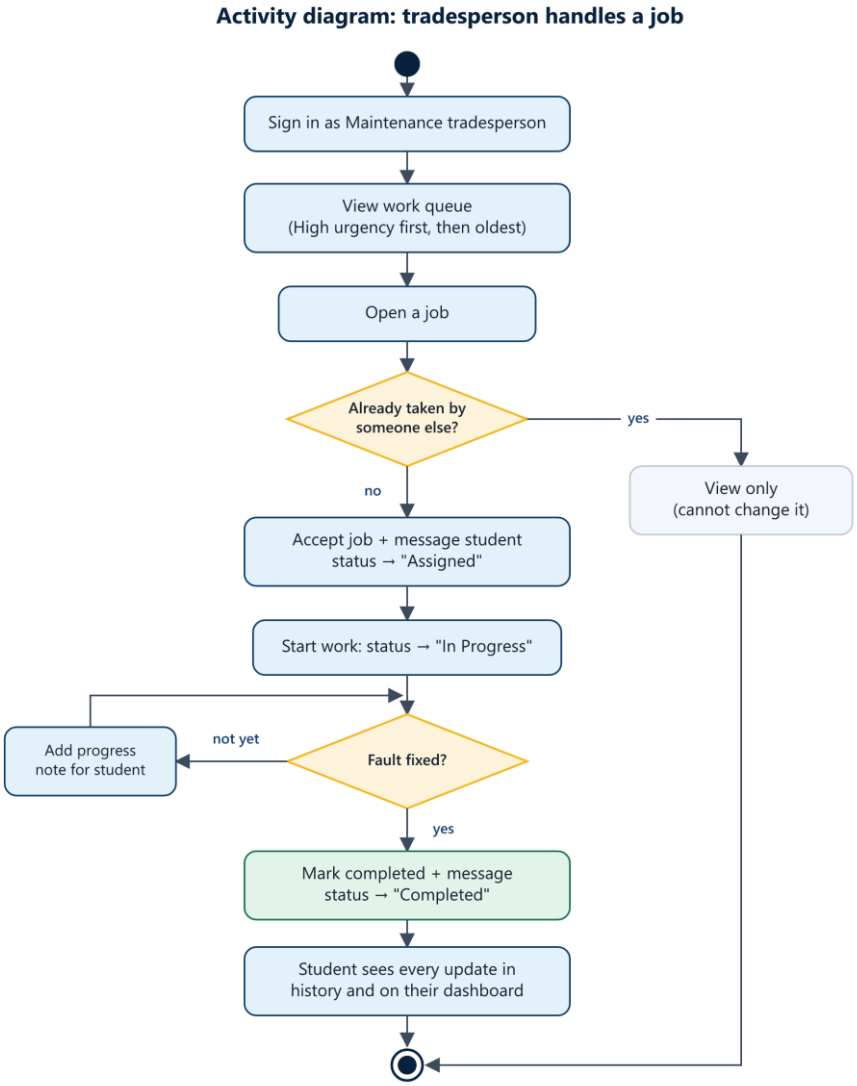



*Figure 5: Activity diagram: tradesperson handles a job*

## 11.3 Sign in and password reset

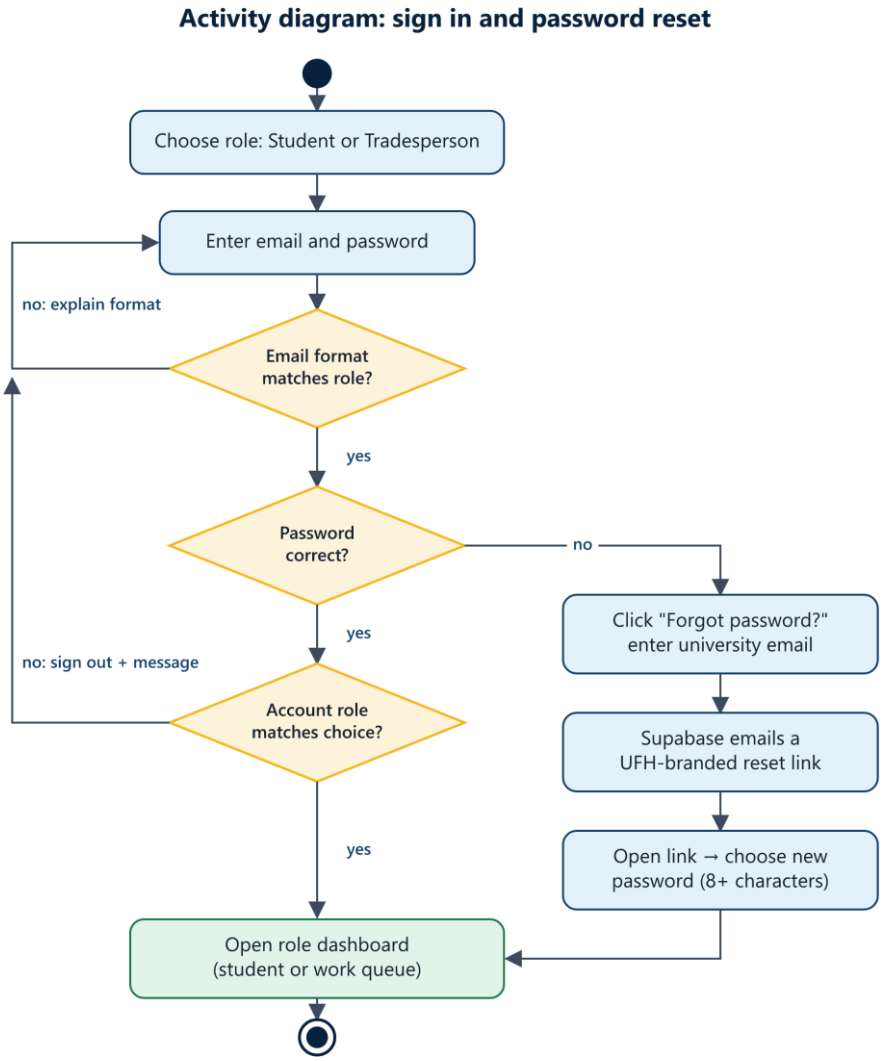



*Figure 6: Activity diagram: sign in and password reset*

# 12. Methodology

**We followed an Agile, iterative and incremental approach.** Instead of designing everything up front, we built a working version early, showed it to the group, collected feedback, and improved it in short cycles. Each change was small, tested, committed to GitHub and, once hosting was set up, deployed automatically.

## 12.1 Iterations

| Iteration | Focus | Outcome |
|---|---|---|
| 1. Prototype | Core idea and data model | Node.js/Express API with SQLite, React screens for logging and tracking faults, a priority algorithm and automated tests. |
| 2. Move to Supabase | Hosted backend | Sign-in, database and photo storage moved to Supabase so the app no longer needs its own server. |
| 3. Features and design | Complete user journeys | Role-based sign-in, password reset, request history, comments, residence list, footer, Contact/Terms/Privacy pages, UFH visual design. |
| 4. Testing and feedback | Quality | Group testing rounds; each round's feedback became fixes (see section 13.4). |
| 5. Deployment | Live system | Hosted on Vercel with automatic deployment from GitHub. |

## 12.2 Team practices

- **Version control:** all work is committed to a shared GitHub repository, with co-author credit for members who worked on each change.
- **Small, described commits:** every commit does one thing and explains why, which makes changes easy to review and undo.
- **Build check before commit:** the production build is run before each commit so broken code is not pushed.
- **Continuous deployment:** Vercel publishes each push to main, so the group always tests the latest version.
- **AI-assisted development:** an AI coding assistant (Claude Code) was used as a pair programmer for implementation, review and diagrams; its contributions are credited in the commit history.

# 13. Demo and Testing

## 13.1 Demonstration plan

The demo uses the live system at **https://ufh-residence-maintenance.vercel.app** with one student account and one tradesperson account, ideally on a phone and a laptop side by side.

1. Show the sign-in page, choose Student, and show that a staff email is refused with a helpful message.
2. Sign in as the student and log a High-urgency fault with a phone photo.
3. On the laptop, sign in as the tradesperson and show the fault at the top of the work queue.
4. Accept the job with a visit message, start work, add a progress note, and mark it completed.
5. Back on the phone, show the student's history and "Latest updates" filling in step by step.
6. Show "Forgot password?" sending a reset link, and the Contact, Terms and Privacy pages.

## 13.2 Automated tests (prototype backend)

The first iteration's Node.js API has 30 automated tests covering authentication, requests, security (including SQL injection and role checks) and the priority algorithm. All 30 passed at the time they were written (see TESTING.md in the repository).

## 13.3 Functional tests of the final system

| ID | Test | Expected result | Result |
|---|---|---|---|
| T-01 | Sign in as Student with a staff email (123456@staff.ufh.ac.za) | Refused before contacting the server, with a message to choose Maintenance tradesperson | Pass |
| T-02 | Email format checks: 8 examples (valid and invalid, both roles, mixed case) | Each accepted or rejected with the correct message | Pass (8/8) |
| T-03 | Sign in with a role that does not match the account | Signed out again with a clear message; wrong dashboard never shown | Pass |
| T-04 | Staff account whose role is "tradesperson" rather than "maintenance" | Recognised as staff and given the work queue | Pass |
| T-05 | Attach a 9 MB, 4000×3000 photo | Resized to 1600×1200, about 0.5 MB, before upload | Pass (520 KB) |
| T-06 | Attach a file that is not an image | "That file is not a photo" message | Pass |
| T-07 | Student opens My requests | Only their own requests are listed | Pass (after fix) |
| T-08 | Tradesperson adds a progress note | Note text shown in the history with name and time | Pass (after fix) |
| T-09 | Open the site at 375 px phone width | No sideways scrolling; buttons and cards fit | Pass |
| T-10 | Open /reset-password directly on the live site | App loads (no 404) and explains the link has expired | Pass |
| T-11 | Production build before every commit | Build completes with no errors | Pass |

## 13.4 User acceptance testing

Group members used the system as students and as tradespeople and reported what did not work or did not feel right. Each piece of feedback led to an improvement, which shows the testing was effective:

| Feedback from testing | Improvement made |
|---|---|
| "I can't tell the portal whether I am a student or a tradesperson." | Added a role choice on the sign-in page, with separate email formats and a role check. |
| "It says invalid login credentials and there is no way to reset my password." | Added Forgot password with an emailed reset link and a clear "email and password do not match" message. |
| "I chose tradesperson but it says my account is a tradesperson." | Found that staff roles were stored under different names; one shared role rule now recognises all of them. |
| "Photo upload fails and it only allows 2 MB." | Photos are now resized automatically, and a storage policy allows students to upload to their own folder. |
| "The history only shows names and dates, not what the tradesperson wrote." | Fixed the history so every note, message and status change is shown, and added student comments and dashboard updates. |
| "The residences listed are wrong." | Replaced the list with Student Village, Marikana, Wesley, ZK and Taunyane, plus Other with a typed name. |
| "The half-and-half sign-in page looks like every other site." | Redesigned it as a full-screen campus photo wall with a centred card. |
| "I don't want a code, just a link." | Password reset changed to a link-only flow with a UFH-branded email. |
| "The sign-in card is too small and the photos are hard to see." | Made the card larger, lightened the overlay so the photos show, and added a subtle 3D tilt. |
| "The emergency numbers on Contact us are not part of this project." | Removed them; Contact us now shows campus addresses and switchboards only. |

# 14. Conclusion

The Residence Maintenance Portal replaces paper forms and word-of-mouth with a single online system where UFH students report residence faults and follow each repair, and where tradespeople see what matters most and keep students informed. It meets the functional requirements set out in section 6, runs on phones and computers, and is live for demonstration.

Building it iteratively, with regular testing by the group, meant that problems were found early and every round of feedback made the system better, from the sign-in experience to how clearly the request history communicates progress.

## 14.1 Limitations and future work

- Tighten database row-level security on maintenance tickets so the rule "students see only their own requests" is also enforced by the database, not only by the app.
- Store user roles where only administrators can change them (for example Supabase app_metadata).
- Send emails from a university address through a dedicated email provider instead of Supabase's default sender.
- Connect the staff dashboard page and add reports for Residence Life, such as the most common faults per residence.
- Add push or SMS notifications for urgent faults.